EuroSAT MobileNetV2: deep structured pruning, teacher-guided recovery, static INT8

Team: Valiant Joy **Model:** Valiant Joy **Submission:** `valiant_joy_Model.onnx` (input `(1, 3, 224, 224)`, ONNX opset 13, static QDQ, 0.60 MB)

Pipeline

1. **Start:** the provided `student_mobilenetv2_distilled.pt` (MobileNetV2, 2.24 M parameters, FP32).
2. **Structured pruning, 60% of channels.** Channels are removed, not masked: the exported graph keeps MobileNetV2's topology (52 convolutions, 17 depthwise, 10 residual adds) with fewer channels, so parameters, file size and MACs all really shrink.
3. **Recovery:** BatchNorm re-estimation, then 8 epochs of fine-tuning distilled from the provided ResNet-50 teacher (KD temperature 4, alpha 0.7).
4. **Static INT8:** QDQ, per-channel weights, percentile calibration on training images; exported with the TorchScript exporter at opset 13 and the input pinned to `(1, 3, 224, 224)`.

Results (`metrics.json`: same machine, same script, same thread count for baseline and final)

	Baseline (provided student)	Final	Ratio
Top-1 accuracy	99.09%	97.15%	98.04% retained
Worst-class accuracy (of 10 recalls)	94.66%	92.61%	-2.05 pt
Parameters	2,236,682	400,954	5.58x fewer
File size	8.77 MB	0.60 MB	14.5x smaller
MACs	326.2 M	79.5 M	4.10x fewer
Latency, 4 threads (1 thread)	3.50 ms (9.80 ms)	1.12 ms (2.68 ms)	3.13x (3.66x)
Peak RAM, raw process RSS	78.9 MB	62.1 MB	1.27x lower
Energy per inference (CodeCarbon, TDP mode)	0.2177 J	0.0709 J	3.07x lower
AI Hub, Dragonwing RB3 Gen 2 (QCS6490), ONNX on CPU	27.97 ms, 32.3 MB	3.13 ms, 19.5 MB	8.94x faster, 1.66x less memory

Machine: i7-13620H, 16 GB, Windows 11, CPU only, ONNX Runtime 1.30.0, 4 intra-op threads. Energy is CodeCarbon's TDP estimate (no power sensor on that laptop), so we checked it against physics: implied power is 62.2 W for the baseline and 63.4 W for the final model, and the 3.07x energy ratio matches the 3.13x latency ratio. AI Hub profile jobs: `jp3o60znp` (baseline), `jprxwv90p` (final).

Cross-check on a second machine (`verification/CROSS_CHECK.md`): the same ONNX file re-measured on an AMD Ryzen 7 4800H (no AVX-VNNI) with an independently generated, stratified 5,400-image split and **measured** energy (CodeCarbon `windows_emi`): 97.1% of accuracy retained (worst class 91.8%), 5.63x fewer parameters and 14.07x smaller (counted from the ONNX graph), 1.86x faster, 1.21x lower peak RAM, 2.21x less energy. The speed-up depends on the hardware: 1.86x on that CPU, 3.13x on the i7-13620H, 8.94x on the QCS6490 edge CPU.

What we tried, kept and discarded

- **Measure the weakest class, not only the mean.** Sweeping the pruning ratio from 0% to 60%, every point up to 20% sits inside the +-0.36-point measurement band of the mean; the worst-class recall is the statistic that moves, and it located the cliff between 50% and 60%. - **Spend on recovery, not the cut.** At 60%, teacher-guided recovery lifted the weakest class by 7.8 points over hard-label recovery, for a few lines of configuration; at 50% it made no difference. The collapse at 60% is a recovery limit rather than a capacity limit, which is what let us ship the deeper point. - **Profile on the target.** The same two ONNX files read 3.13x on our laptop and 8.94x on the edge CPU: a large out-of-order core with a deep cache hides most of what compression buys, so choosing a ratio from laptop latency alone would read the gradient about three times too shallow. - **Discarded: per-layer ratio allocation.** Better at matched size, but it stalls at 46.6% overall, because the cheap parameters are a bounded pool. - **The cost we report:** the weakest class falls to 92.6%, below the baseline's 94.7% (on the second-machine check it is PermanentCrop, the class that is also weakest for the baseline). One class carries most of the loss; we report it rather than average it away.

Reproducibility

CODE_SECTION_PENDING